IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (Deemed to be University)

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

A Constituent Unit of Yenepoya (Deemed to be University)

Balmatta, Mangalore - 575002

Project – High Level Design

On

HARDENED CLOUD INFRASTRUCTURE FOR RETAIL DATA

Student Name: Bhavana Pradeep

Industry Mentor: Mr.shashank

Guided By: Ms.Munisah

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1. Scope of the Document 1
1.2. Intended Audience 2
1.3. System Overview 2
2. SYSTEM DESIGN 3
2.1. Application Design 3
2.2. Process Flow 3
2.2.1 File Upload Process Flow (Encryption Pipeline) 3
2.2.2 File Download Process Flow (Decryption Pipeline) 5
2.3. Information Flow 5
2.4. Components Design 6
2.5. Key Design Considerations 7
2.5.1 Zero-Trust Storage Principle 7
2.5.2 AES-256-GCM over AES-256-CBC 7
2.5.3 PBKDF2 Key Derivation at 600,000 Iterations 7
2.5.4 UUID-Based File Naming 7
2.5.5 Tamper-Evident Audit Trail 7
2.6. API Catalogue 7
3. DATA DESIGN 9
3.1. Data Model 9
3.1.1 User Table 9
3.1.2 FileRecord Table 9
3.1.3 AuditLog Table 10
3.2. Data Access Mechanism 10
3.3. Data Retention Policies 10
3.4. Data Migration 10
4. INTERFACES 11
4.1. User Interface — Crypto Vault 11
4.2. Admin Interface — Admin Dashboard 13
5. STATE AND SESSION MANAGEMENT 13
6. CACHING 14
7. NON-FUNCTIONAL REQUIREMENTS 15
7.1. Security Aspects 15
8. REFERENCES 16

1. INTRODUCTION

This document presents the High-Level Design (HLD) of the Hardened Cloud Infrastructure for Secure Retail Data Storage system. The system is architected to provide military-grade cryptographic protection for sensitive retail data including customer personally identifiable information (PII), payment transaction logs, and corporate contracts stored on a server-based cloud infrastructure. The primary innovation of this project lies in the absolute guarantee that no data ever touches the storage medium in plaintext form, thereby neutralising the threat of data breaches resulting from physical server compromise or insider attacks.

The retail sector has witnessed an exponential surge in digitisation, making cloud-hosted data repositories the industry norm. However, conventional cloud storage solutions apply encryption inconsistently, leaving data-at-rest vulnerable to sophisticated internal threats. This system addresses this critical gap through a zero-trust storage model enforced by a standalone AES-256-GCM encryption engine integrated at the application layer.

1.1. Scope of the Document

This HLD document covers the following aspects of the system:

- Overall system architecture and the interaction between its four primary modular layers
- Application design including the process flow of encryption and decryption operations
- Information flow from the client browser through the application gateway to the encrypted storage layer
- Component design detailing the responsibilities of each system module
- Key design considerations including the zero-trust storage principle and cryptographic strategy
- API Catalogue of all RESTful endpoints exposed by the Flask application gateway
- Data design including the SQLite database schema and data access mechanisms
- Interface specifications for the frontend user interface
- State and session management strategy
- Caching considerations
- Non-functional requirements covering security and performance aspects

This document does not cover low-level implementation details such as line-by-line code logic, which are addressed in the companion Low-Level Design (LLD) document.

1.2. Intended Audience

This document is intended for the following stakeholders:

- Academic evaluators and faculty reviewing the final year project submission
- Developers or students who may extend or build upon this system in future iterations
- Cloud security architects seeking reference implementations of zero-trust storage models
- Information security professionals interested in practical AES-256-GCM application layer encryption
- Anyone studying data-at-rest security mechanisms for retail and enterprise environments

1.3. System Overview

The Hardened Cloud Infrastructure for Secure Retail Data Storage is a server-based web application built using Python and the Flask framework. The system processes file uploads submitted by authenticated users, encrypts the file contents in memory using AES-256 in Galois/Counter Mode (GCM) with a PBKDF2-derived key, and persists only the resulting ciphertext to the local file system. The original plaintext never contacts the storage disk.

The architecture is separated into four distinct modular layers as shown in the table below:

Component	Technology	Purpose
Client View (Frontend UI)	HTML, CSS, JavaScript (Vanilla)	User interaction, file upload/download via RESTful AJAX calls
Application Gateway	Python, Flask, Flask-Login	Session management, authentication, role-based access control, request routing
Cryptographic Engine	Python cryptography library, PBKDF2HMAC, AES-256-GCM	Key derivation, encryption, and decryption of file byte streams
Storage Layer	Local File System + SQLite (via SQLAlchemy ORM)	Encrypted ciphertext persistence and cryptographic metadata management

1

2.SYSTEM DESIGN

2.1. Application Design

The system is constructed as a multi-page Flask web application that enforces a strict separation of concerns across its four layers. The client-side interface communicates with the Flask backend exclusively through defined RESTful API endpoints using AJAX-based HTTP requests. The backend intercepts every file operation, routing the file byte stream through the Core Cryptographic Engine before any data reaches the storage subsystem. This architectural decision ensures that the encryption boundary is consistently enforced regardless of the client or network condition.

The application uses a Role-Based Access Control (RBAC) model managed by Flask-Login. Two roles are defined: User and Admin. Regular users have access to their own Crypto Vault, which displays, uploads, downloads, and deletes their own encrypted files. Administrators additionally have access to the Admin Dashboard, which displays system-wide metrics, user management functions, and the complete System Audit Trail.

The frontend is designed using a Glassmorphism aesthetic with drag-and-drop upload zones. The user interface dynamically communicates encryption status to the user and warns clearly that lost encryption passwords cannot be recovered, enforcing the zero-knowledge property of the storage design.

2.2. Process Flow

The process flow of the system follows two primary pipelines corresponding to the two core operations: File Upload (Encryption) and File Download (Decryption).

2.2.1 File Upload Process Flow (Encryption Pipeline)

The following sequence describes the complete encryption pipeline:

- Step 1 — Authentication Check: The Flask application gateway verifies that the requesting user has an active, valid session. Unauthenticated requests are redirected to the login page.
- Step 2 — Password Capture: The server captures the encryption_password field from the multipart form POST payload. This password is never stored anywhere in the system.
- Step 3 — File Byte Extraction: The uploaded file is read entirely into server memory as a byte stream. The file does not touch the disk at this stage.
- Step 4 — Integrity Snapshot: A SHA-256 hash of the raw plaintext byte stream is computed and stored as the file_hash field. This enables future integrity verification.

- Step 5 — Cryptographic Engine Invocation: CoreCryptoEngine.encrypt_data(bytes, password) generates a random 32-byte salt and 12-byte nonce, then executes PBKDF2HMAC with SHA-256 and 600,000 iterations to derive a 256-bit symmetric key. AES-256-GCM encryption is performed using this key and nonce.
- Step 6 — Ciphertext Storage: The resulting ciphertext blob is written to /storage/<user_id>/<uuid>. UUID naming prevents path traversal attacks.
- Step 7 — Metadata Persistence: The salt (hex), nonce (hex), original filename, UUID, user_id, and file_hash are written to the FileRecord table in SQLite.
- Step 8 — Audit Log Entry: An AuditLog record is created capturing user_id, action (UPLOAD), status, IP address, and timestamp.

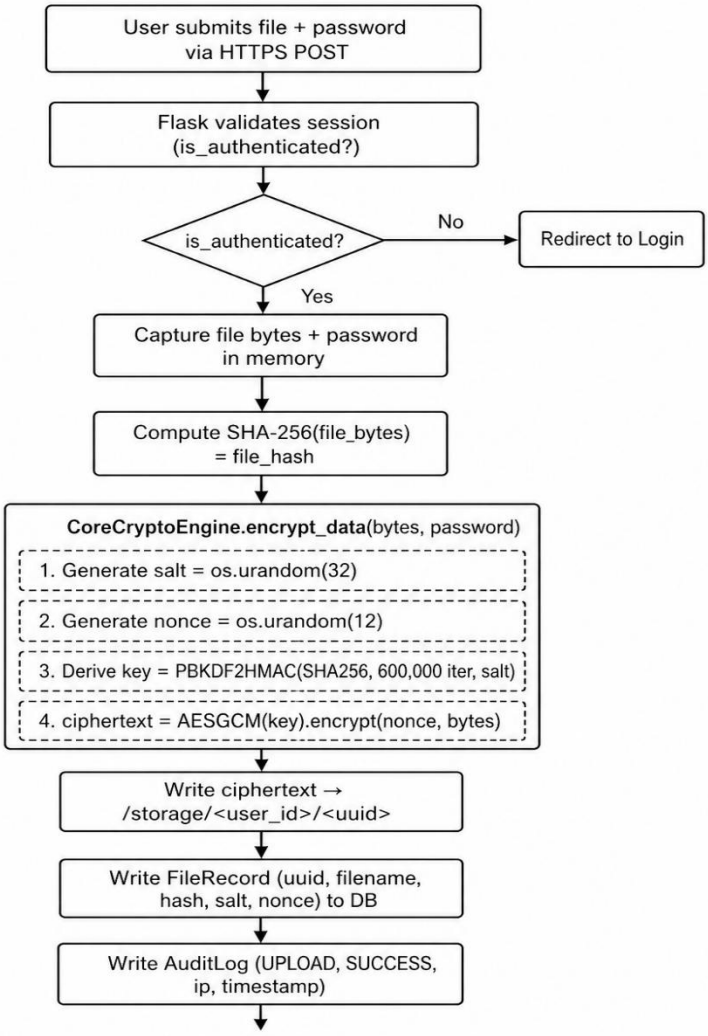

Figure 2.1 — File Upload (Encryption) Process Flow

2.2.2 File Download Process Flow (Decryption Pipeline)

- Step 1 — Authentication and Ownership Check: The server verifies that the logged-in user owns the requested file_id.
- Step 2 — Metadata Retrieval: The FileRecord entry is queried from SQLite to obtain the UUID-named ciphertext file path, salt, and nonce values.
- Step 3 — Ciphertext Loading: The encrypted ciphertext blob is read from the file system into server memory.
- Step 4 — Password Capture: The decryption_password is captured from the POST payload.
- Step 5 — Cryptographic Engine Invocation: CoreCryptoEngine.decrypt_data(ciphertext, password, salt, nonce) re-derives the PBKDF2 key using the stored salt. AES-GCM performs authenticated decryption. If the GCM tag fails, a ValueError is raised.
- Step 6 — Plaintext Delivery: The decrypted byte stream is returned directly to the client as a send_file response in memory. The plaintext never touches disk.
- Step 7 — Audit Log Entry: An AuditLog record is created for the DOWNLOAD action.

2.3. Information Flow

Data flows through the system in the following strictly ordered direction, ensuring the encryption boundary is never bypassed:

Stage	From	To	Data Type
1	Browser Client	Flask Application Gateway	HTTPS POST: FormData (file bytes + password)
2	Flask Gateway	Core Cryptographic Engine	Raw plaintext byte stream + password string
3	Core Cryptographic Engine	Flask Gateway	Ciphertext blob + salt (hex) + nonce (hex)
4	Flask Gateway	File System (/storage/)	Ciphertext blob written to UUID-named file
5	Flask Gateway	SQLite Database	File metadata: filename, UUID, salt, nonce, file_hash, user_id
6	Flask Gateway	SQLite Database	Audit log entry: user, action, IP, timestamp, status
7 (Download)	File System	Core Cryptographic Engine	Ciphertext blob + salt + nonce + password

Stage	From	To	Data Type
8 (Download)	Core Cryptographic Engine	Flask Gateway	Decrypted plaintext byte stream (in memory)

2.4. Components Design

The system is composed of the following major components, each with a well-defined responsibility boundary:

Component	Description
Client View (Frontend UI)	Built with Vanilla HTML, CSS, and JavaScript. Implements a Glassmorphism aesthetic with drag-and-drop upload zones. Communicates with the backend exclusively via RESTful AJAX using FormData.
Application Gateway (Flask Backend)	Serves as the central router and access controller. Employs Flask-Login for session management. Enforces role-based access control. Intercepts all file uploads and downloads to route them through the Cryptographic Engine.
Core Cryptographic Engine (security/core_crypto.py)	The absolute security core. Accepts a password and byte stream as input. Uses PBKDF2HMAC-SHA256 (600,000 iterations) to derive a 256-bit AES key. Generates cryptographically random salt and nonce. Performs AES-256-GCM encryption and authenticated decryption. Raises ValueError on authentication failure.
Intrusion Prevention System (IPS)	Monitors the /auth/login endpoint. Tracks failed login attempts per user via the failed_attempts column. Locks the account for 15 minutes upon exceeding five consecutive failed attempts.
Storage Layer (File System)	Stores only ciphertext blobs under /storage/<user_id>/ using UUID-based filenames. UUIDs prevent path traversal attacks and metadata leakage.
Database Layer (SQLite via SQLAlchemy)	Stores only metadata — never plaintext file contents. Three primary models: User, FileRecord, and AuditLog.
Admin Dashboard	Accessible only to Admin-role users. Displays total users, encrypted files stored, and failed login attempt counts. Presents the complete System Audit Trail.
Cryptographic Shredding Module	Executed on file deletion. Overwrites the ciphertext file with os.urandom() bytes before calling os.remove(), preventing forensic data recovery.

2.5. Key Design Considerations

2.5.1 Zero-Trust Storage Principle

The foundational design principle is that the storage layer is treated as an inherently untrusted medium. No process within the system has the capability to write plaintext data to the file system. Encryption occurs atomically in server memory before any disk write.

2.5.2 AES-256-GCM over AES-256-CBC

Galois/Counter Mode (GCM) was selected over CBC mode because GCM provides authenticated encryption. The decryption process verifies the integrity and authenticity of the ciphertext before producing any plaintext output, defending against bit-flipping attacks.

2.5.3 PBKDF2 Key Derivation at 600,000 Iterations

PBKDF2HMAC with SHA-256 at 600,000 iterations, aligned with NIST SP 800-132 (2023) recommendations, makes brute-force attacks computationally infeasible. A unique random salt per file prevents rainbow table attacks.

2.5.4 UUID-Based File Naming

All stored ciphertext files are named using UUIDs generated at upload time. This prevents path traversal attacks, filename enumeration attacks, and metadata leakage through the file system.

2.5.5 Tamper-Evident Audit Trail

The AuditLog table is an append-only ledger of all significant system events, accessible only to administrators. It provides visibility into all upload, download, login, and deletion events with IP address and timestamp.

2.6. API Catalogue

Authentication Endpoints

- /auth/register (GET, POST)
  - No authentication required
  - Displays registration form (GET)
  - Handles new user account creation (POST)
- /auth/login (GET, POST)
  - Authenticates user credentials
  - Includes IP-based rate limiting (5 attempts, 15-minute lockout)
- /auth/logout (GET)
  - Authentication required
  - Ends user session
  - Redirects to login page

File Management Endpoints

- /files/upload (POST)
  - Authentication required
  - Accepts file upload with encryption password
  - Encrypts file using AES-256-GCM

- /files/download/<file_id> (POST)
  - Authentication required
  - Requires decryption password
  - Returns decrypted file as downloadable content
- /files/delete/<file_id> (POST)
  - Authentication required
  - Performs cryptographic shredding of file
  - Deletes associated file record permanently

User Dashboard

- /dashboard (GET)
  - Authentication required (User)
  - Displays Crypto Vault
  - Shows list of user's encrypted files

Admin Endpoints

- /admin/dashboard (GET)
  - Authentication required (Admin)
  - Displays system metrics
  - Provides audit trail
- /admin/users (GET)
  - Authentication required (Admin)
  - User management interface
  - Allows administrative operations on users

3.DATA DESIGN

3.1. Data Model

The backend database is SQLite, managed through the SQLAlchemy ORM. Three primary model classes represent the core entities of the system. The data model ensures the database never stores plaintext file content; it stores only the metadata required for cryptographic operations and system auditing.

3.1.1 User Table

Column	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Unique user identifier
username	Varchar(80)	Unique, Not Null	User's login username
password_hash	Varchar(128)	Not Null	Bcrypt hash of the user's account password
role	Varchar(10)	Not Null, Default: 'user'	Access role: 'user' or 'admin'
failed_attempts	Integer	Default: 0	Count of consecutive failed login attempts for IPS logic
locked_until	DateTime	Nullable	Timestamp until which the account is locked by the IPS

3.1.2 FileRecord Table

Column	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Unique file record identifier
user_id	Integer	FK -> users.id, Not Null	References the owning user
filename	Varchar(36)	Not Null	UUID-based ciphertext filename on disk
original_filename	Varchar(255)	Not Null	Original user-supplied filename for download presentation
file_hash	Varchar(64)	Not Null	SHA-256 hex digest of pre-encryption plaintext for integrity verification
salt	Varchar(64)	Not Null	Hexadecimal representation of the random 32-byte PBKDF2 salt

3.1.3 AuditLog Table

Column	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment	Unique log entry identifier
user_id	Integer	FK -> users.id, Nullable	References the acting user (null for system events)
action	Varchar(20)	Not Null	Action type: UPLOAD, DOWNLOAD, LOGIN, LOGOUT, DELETE
status	Varchar(10)	Not Null	Outcome: SUCCESS or FAILURE
ip_address	Varchar(45)	Not Null	Client IP address at the time of the action
timestamp	DateTime	Not Null, Default: UTC Now	Precise UTC timestamp of the event

3.2. Data Access Mechanism

- All database queries are performed through SQLAlchemy's session-based query API, preventing raw SQL injection vulnerabilities.
- The User model is loaded per request by Flask-Login's user_loader callback, which queries by primary key.
- FileRecord entries are always queried with an additional filter on user_id to enforce ownership boundaries. No user can access another user's file records.
- AuditLog entries are written using append operations and are never deleted through the normal application lifecycle.

3.3. Data Retention Policies

- Encrypted ciphertext files are retained on the file system until the owning user explicitly performs a deletion operation, which invokes the cryptographic shredding routine.
- FileRecord entries are deleted from the database upon file deletion by the owning user.
- AuditLog entries are retained indefinitely as a tamper-evident system audit record, accessible only to Admin-role users.
- User account records are retained until an administrator performs an explicit user deletion from the User Management panel.

3.4. Data Migration

The current project scope uses SQLite as the primary data store. The SQLAlchemy ORM layer abstracts the database dialect, enabling future migration to PostgreSQL or MySQL without changes to application-level query logic.

4. INTERFACES

The system provides two distinct web-based interface views through Flask-rendered Jinja2 templates, served over HTTP (HTTPS in production).

4.1. User Interface — Crypto Vault

UI Element	Type	Description
Navigation Sidebar	HTML Nav	Displays user access level badge and navigation links (Crypto Vault, Secure Logout)
Welcome Banner	HTML Section	Displays the authenticated username with tagline 'Your data is cryptographically isolated'
Cryptography Password Field	HTML Input (password)	Accepts the per-file encryption/decryption password. Includes warning that lost passwords cannot be recovered.
Drag-and-Drop Upload Zone	HTML Div + JavaScript	Dashed-border drop zone accepting file selection by drag-and-drop or click-to-browse.
Encrypted Files List	HTML Table	Displays the user's uploaded files with filename, upload timestamp, and action buttons for download and delete.
Download Password Modal	JavaScript Modal	Appears on download button click. Captures the decryption password before submitting the AJAX download request.

Figure 4.1 below shows the SecureCloud User Dashboard (Crypto Vault) as implemented:

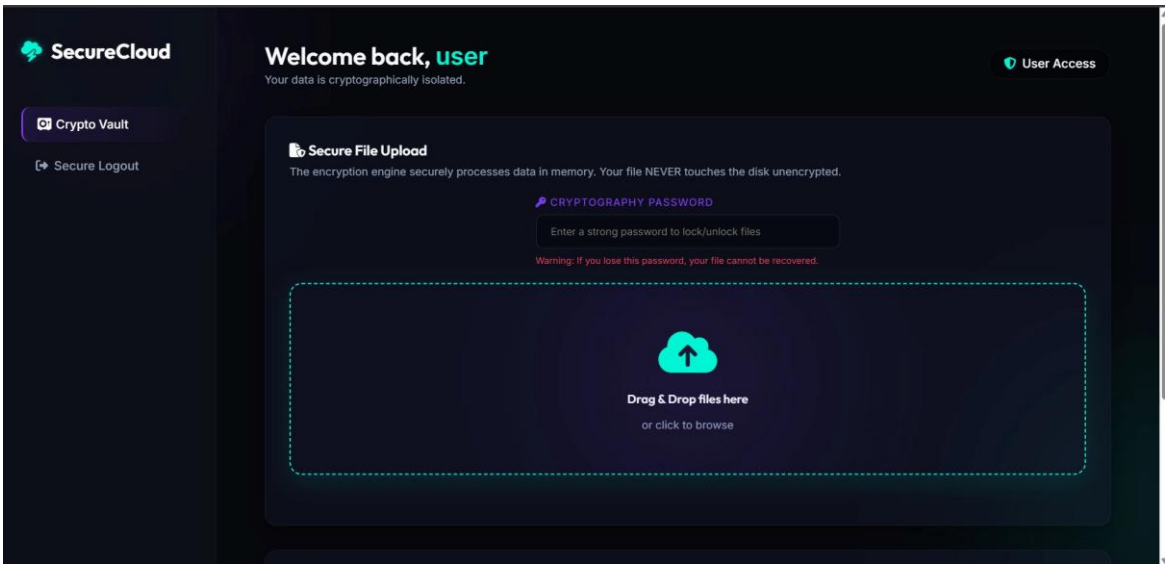

Figure 4.1 — SecureCloud User Dashboard: Secure File Upload with Drag-and-Drop Interface

4.2. Admin Interface — Admin Dashboard

UI Element	Type	Description
Admin Dashboard Metric Cards	HTML Cards	Three summary cards displaying Total Users (purple), Encrypted Files Stored (cyan), and Failed Login Attempts (red).
System Audit Trail Grid	HTML Table	Full-width table showing: Timestamp, User ID, Action, Status (colour-coded), IP Address, and Details.
Users Management Panel	HTML Table	Lists all registered users with role assignments and administrator action buttons.

Figure 4.2 below shows the SecureCloud Admin Dashboard as implemented:

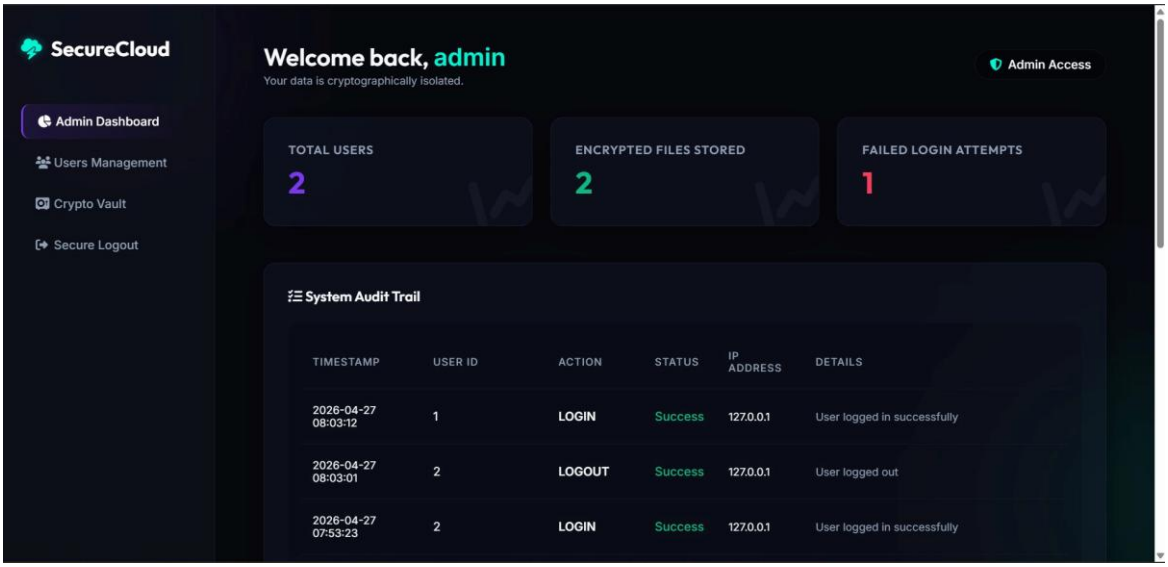

Figure 4.2 — SecureCloud Admin Dashboard: System Metrics and Audit Trail

5.STATE AND SESSION MANAGEMENT

The system uses Flask-Login for server-side session management. Upon successful authentication, Flask-Login establishes a signed session cookie that persists across requests during the browser session. The session cookie stores the user's unique ID and is cryptographically signed using the Flask application's SECRET_KEY, preventing tampering.

All protected routes are decorated with @login_required, which enforces that only authenticated users can access application functionality. Role-based access is enforced at the route handler level by checking the current_user.role attribute and returning HTTP 403 for unauthorised role access.

The IPS logic maintains per-user account lock state in the User table itself (failed_attempts and locked_until columns) rather than in the session, ensuring lock state persists across multiple sessions and IP address changes. There is no client-side session state storage; all authoritative state is maintained server-side.

6.CASHING

The current project scope does not implement application-level caching, as the primary security requirement mandates that all file data be processed through the encryption engine on every operation. Caching plaintext data at the application level would violate the zero-trust storage principle.

Flask's built-in development server does not implement HTTP response caching for dynamic routes. Static assets (CSS, JavaScript) may be served with standard HTTP cache-control headers by the web server in a production deployment. Sensitive dynamic responses (file downloads, API responses) are served with Cache-Control: no-store headers to prevent browser caching of decrypted content.

7.NON-FUNCTIONAL REQUIREMENTS

7.1. Security Aspects

- Data-at-Rest Encryption: All stored files are encrypted with AES-256-GCM prior to any disk write operation. Plaintext data never reaches the storage medium.
- Password-Based Key Derivation: PBKDF2HMAC with SHA-256 and 600,000 iterations derives the AES key. A unique random 32-byte salt per file prevents rainbow table attacks.
- Authenticated Encryption: AES-GCM mode provides both confidentiality and integrity verification. Bit-flipping and tampering attacks are detected and rejected.
- Cryptographic Shredding: File deletion overwrites ciphertext with random bytes before unlinking the file, preventing forensic data recovery.
- Intrusion Prevention System: Five consecutive failed login attempts trigger a 15-minute account lockout, defeating brute-force and credential-stuffing attacks.
- Bcrypt Password Hashing: User account passwords are hashed using Bcrypt, ensuring stored credentials cannot be reversed.
- Role-Based Access Control: Admin functionality is restricted to users with the Admin role. All other users have access only to their own data.
- UUID File Naming: Ciphertext files are stored with UUID-based names, preventing path traversal and filename enumeration attacks.
- Tamper-Evident Audit Trail: All security-relevant events are logged with IP address, timestamp, and outcome. The log is viewable only by administrators.

7.2. Performance Aspects

- In-Memory Processing: File byte streams are processed entirely in server memory, eliminating intermediate plaintext disk writes and reducing I/O latency.
- PBKDF2 Iteration Trade-off: The 600,000 iteration count introduces approximately 0.3 to 0.8 seconds of computation per encryption/decryption operation. This is acceptable for file operations and serves as an inherent brute-force deterrent.
- SQLAlchemy Connection Pooling: The ORM manages a connection pool to the SQLite database, reducing connection establishment overhead for concurrent requests.
- Static Asset Serving: Static frontend assets are served with appropriate HTTP caching headers to reduce repeated bandwidth usage.

8.REFERENCES

- Python Cryptography Library Documentation — https://cryptography.io/en/latest/
- NIST SP 800-132: Recommendation for Password-Based Key Derivation — https://csrc.nist.gov/publications/detail/sp/800-132/final
- Flask Documentation — https://flask.palletsprojects.com/en/3.0.x/
- Flask-Login Documentation — https://flask-login.readthedocs.io/
- SQLAlchemy ORM Documentation — https://docs.sqlalchemy.org/
- AES-GCM Specification — NIST SP 800-38D — https://csrc.nist.gov/publications/detail/sp/800-38d/final
- Bcrypt Password Hashing — https://pypi.org/project/bcrypt/
- OWASP Cryptographic Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- UUID RFC 4122 — https://www.rfc-editor.org/rfc/rfc4122